

Validator-Guided Repair on a Shared Initial LLM Candidate: A Preregistered Paired Evaluation for Network Configuration Safety

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

Abstract—We preregistered and executed a paired confirmatory experiment (PREREG-CAND-001-VER-GVR-2026-09-01) to measure whether permitting a deterministic network-configuration validator plus one automated repair changes the rate of validator-valid executable configurations relative to leaving the identical sampled LLM candidate unguarded (`shared_initial_candidate` semantics). Using the declared generator `gpt-5-mini` (hosted adapter `openai_responses`) and deterministic `netops_validator_v5` on $N = 200$ paired intents (`completed_episode_count = 400`; `model_calls_used = 200`), every shared initial candidate passed validator checks and no repairs were attempted or applied (`repair_attempt_count = 0`; `repair_applied_count = 0`). Observed outcomes: `baseline_success = 200/200`, `guarded_success = 200/200`; paired contingency $n11 = 200$, $n10 = 0$, $n01 = 0$, $n00 = 0$; exact McNemar two-sided $p = 1.0$; paired bootstrap percentile 95% CI (seed = 1729; 10,000 resamples) = $[0.0, 0.0]$. We present artifact-grounded methodology, execution accounting, representative validator diagnostics, compact contingency and repair summaries, and a focused discussion clarifying the causal limits of the executed estimand under `shared_initial_candidate` semantics and preregistered follow-on designs required to measure repair efficacy under independent sampling, higher difficulty, or adversarial inputs.

I. INTRODUCTION

Generative large language models (LLMs) are increasingly proposed to translate operator intent into device configuration sequences in network operations (NetOps). Erroneous sequences, syntactic mistakes, or fabricated fields can cause service disruption or policy violations when applied to devices. A common architectural mitigation is a generate→validate→repair (GVR) pipeline: generate candidate configuration(s), deterministically validate them against intent/topology/workflow constraints, and trigger automated repair when the validator finds faults. Prior work documents verifier-guided improvements in infrastructure-as-code and related code-generation domains and recommends grounding strategies to reduce hallucination in operational tasks [1–3]. However, whether verifier-guided pipelines reduce execution-level operational risk for network configurations remains an open empirical question because prior demonstrations often measure textual correctness or IaC test suites rather than executed device-state safety in packet-level or emulator contexts [3,4].

This paper reports a fully preregistered paired confirmatory experiment that isolates the downstream causal contribution of a deterministic validator and any permitted repair acting on the

exact same sampled LLM candidate (`shared_initial_candidate` semantics). Under these semantics baseline and guarded episodes for each intent begin from the identical initial generation; any paired difference can therefore only arise from deterministic validation and a permitted repair of that same candidate. Our primary research question is: under `shared_initial_candidate` semantics, does permitting a deterministic validator plus at most one automated repair change the proportion of validator-valid executable configurations relative to leaving the same initial candidate unguarded? The contribution is an artifact-grounded report of the executed estimand with complete execution accounting, exact inferential statistics, representative validator diagnostics, and precise recommendations for preregistered follow-on designs that can measure repair efficacy when independent sampling, elevated difficulty, or adversarial inputs are employed.

II. RELATED WORK

The literature motivating verifier-guided pipelines for operational automation spans documented LLM unreliability, grounding/RAG methods, verifier-guided code/IaC repair systems, and NetOps capability studies. Surveys and empirical analyses consistently identify hallucination and factual unreliability as core risks when applying LLMs to operational tasks; these works motivate grounding and verification layers to limit unsupported or fabricated outputs in high-stakes contexts [1]. Evidence-grounded and retrieval-augmented generation (RAG) designs have been shown to reduce hallucination for operations-style question answering and document retrieval tasks in applied evaluations, improving textual factuality and traceability in AIOps and DevOps scenarios; however, these studies typically measure document/answer correctness rather than network-state executability after applying generated device commands [2].

Generate→validate→repair architectures are an active research direction in adjacent domains. Recent work on infrastructure-as-code (IaC) demonstrates that integrating verifier feedback into generation or fine-tuning loops can reduce failing IaC deployments and improve policy compliance in that domain; these method papers provide a conceptual and empirical foundation for applying verifier-guided repair to other configuration tasks but do not by themselves establish transfer to device-level NetOps execution [3]. Related program-repair and verifier-guided self-correction studies

further show that a verification step plus automated repair can raise textual/program correctness metrics, again typically measured against test suites or formal specifications rather than emulator/execution outcomes. We therefore treat these results as methodological precedent but not as direct evidence that GVR pipelines will prevent unsafe executable network states when generator outputs are enacted on device models or emulators.

NetOps-specific evaluations have studied intent translation and LLM capability for configuration generation, showing promise for mapping operator intent to device commands but emphasizing accuracy metrics and held-out textual correctness rather than standardized execution-level safety or emulator-based validation [4]. Broad AIOps surveys and reviews highlight a persistent gap: many capability benchmarks and capability-oriented evaluations do not include standardized execution-aware safety metrics, and they call for improved validator fidelity and execution-level benchmarks to bridge generation to operational safety [5].

Putting these lines of work together suggests two cautions that guided our design. First, verifier-guided improvements observed in IaC and program-repair domains motivate testing the same architecture in NetOps but do not substitute for execution-level experiments: transferability is an empirical question. Second, experiments that conflate generator sampling variability with downstream validator/repair effects make causal attribution difficult; a clearly specified estimand and sampling semantics are necessary to isolate whether validation/repair changed outcomes for the same generated candidate versus whether improved first-pass sampling reduced errors. These considerations led us to the preregistered `shared_initial_candidate` paired design used in this study, which isolates downstream validator/repair effects on identical generator outputs while acknowledging that independent-sampling designs are needed to evaluate generator-level error reduction.

III. METHODOLOGY

Preregistration and primary estimand. We preregistered the study as PREREG-CAND-001-VER-GVR-2026-09-01 and froze the primary estimand and analysis before any model calls. The primary estimand is the `paired_success_rate_difference_guarded_minus_baseline`: the per-intent paired difference in the proportion of validator-valid executable configurations under `shared_initial_candidate` semantics. The preregistration explicitly set `generation_semantics = "shared_initial_candidate"` (exactly one sampled initial generation per pair), `task_count = 200`, `planned_episode_count = 400`, and `maximum_repair_calls_per_task = 1`. The primary statistical test specified was the exact two-sided McNemar test, accompanied by a paired bootstrap percentile 95% confidence interval (`bootstrap_seed = 1729`; `bootstrap_resamples = 10000`) for the paired difference.

Task bank, deterministic generation, and contamination controls. A deterministic task generator produced a man-

ifest of 200 synthetic `intent_configuration_repair_v1` tasks. Each manifest entry records `initial_state`, `required_sequence`, difficulty annotations (e.g., "very_hard", "extreme"), and a `generation_seed` to permit deterministic replay. Preexecution contamination checks were performed per the preregistration: SHA-256 exact-match detection across repair and corpus artifacts and n-gram similarity screening. No exact-match contamination exclusions were raised for the confirmatory sample; contamination artifacts and the task manifest are archived in the evidence bundle for independent verification. The preregistration defined explicit exclusion rules (exact-match exclusions moved to an exploratory leaked bucket) and a missingness policy (`complete_pair_primary`) for handling simulator or execution failures.

Model calls, sampling semantics, and provider audit. Execution used the declared generator labeled `gpt-5-mini` via the hosted adapter `openai_responses`. The preregistered execution contract required one shared initial generation per pair; the execution manifest records `model_calls_used = 200`, `hosted_model_call_event_count = 200`, `completed_hosted_model_call_event_count = 200`, and `cache_miss_count = 200`. The archived `model_configuration.json` declares `requested_model = "gpt-5-mini"` and `temperature = null/unset`; the preregistration and manuscript explicitly note that `null/unset` is not evidence of deterministic model sampling and does not imply `temperature = 0`. Provider audit metadata (`stage_counts` showing `shared_initial_generation = 200` and no cross-condition cache reuse) were checked as part of deterministic reconciliation to confirm the intended `shared_initial_candidate` semantics were followed.

Deterministic validator, scoring rules, and repair policy. The study used `deterministic_netops_validator_v5` with `scoring_rule = "full_intent_constraint_validation."` The validator ingests the generated candidate commands, parses them into structured per-interface field updates, applies the task's `workflow_policies` and `required_sequence` constraints, and produces a structured validator trace. Each per-episode scoring artifact includes fields such as `normalized_configuration`, `parsed_command_count`, `required_command_count`, `observed_touched_field_count`, `violation_codes`, `violation_count`, `validator_id/version`, `validation_before` and `validation_after` subtables, a `repair_applied` boolean flag, and a score with a `score_reason_code` (for example, "VALID_CONFIGURATION"). In guarded episodes the policy permitted at most one automated repair attempt (`maximum_repair_calls_per_task = 1`); if invoked, the repair would produce a revised candidate that the validator would revalidate and record in the trace. Under `shared_initial_candidate` semantics the same initial sample was provided to both baseline and guarded episodes; only the guarded arm could alter that identical candidate via the permitted repair.

Outcome mapping, missingness, and multiplicity. `Validator.valid` (`validator.valid == true` in the `validation_after` subtable) was mapped to a binary "executable/valid" outcome for

each episode. The preregistration defined the primary missingness policy `complete_pair_primary` (exclude incomplete pairs from the confirmatory McNemar analysis) and a sensitivity plan that treats any missing or failed episode as unsafe (failure-as-zero). Multiplicity control: a single confirmatory McNemar test was preregistered (no multiplicity correction for the primary test); exploratory subgroup analyses (e.g., by declared difficulty pattern) were labeled exploratory and, if reported, were to be corrected using Benjamini–Hochberg FDR at 0.05.

Deterministic reconciliation and audit checks. The methodology required deterministic reconciliation of episode accounting after execution. Reconciliation checks included verifying `planned_episode_count = 400`, `completed_episode_count` equality, symmetry of `shared_initial_generation` counts, `provider_call_audit` totals (`hosted_model_call_event_count` and cache statistics), and consistent pairing (`complete_pair_count = task_count`). All deterministic consistency checks (provider call audit, episode accounting, and pair accounting) were preregistered as required for the primary analysis to proceed. The analysis executor was instructed to read raw scoring artifacts and compute the paired contingency table exactly from per-episode `validator.valid` outcomes; bootstrap resampling used the archived seed and resample count for reproducibility.

Analysis pipeline. The analysis plan specified `paired_binary_analysis_v1` to compute the contingency table (n_{11} , n_{10} , n_{01} , n_{00}), run the exact two-sided McNemar test, and compute a paired bootstrap percentile 95% CI (`bootstrap_seed = 1729`; `resamples = 10000`) for the paired difference. The analysis artifacts generated (`analysis/condition_summary.csv`, `analysis/paired_contingency_table.csv`, `analysis/missingness_summary.csv`) were archived and hashed. Any deviations from the preregistration (for example, contamination exclusions or incomplete pairs) would be documented and reported; no such deviations occurred in this run. The methodology explicitly avoided human adjudication of validator outcomes: per preregistration, all scoring was deterministic and machine-recorded, and the primary analysis used those machine labels without human relabeling.

IV. RESULTS

Execution accounting and deterministic reconciliation. The execution completed exactly as preregistered: `planned_episode_count = 400`, `completed_episode_count = 400`, `failed_episode_count = 0`, and `model_calls_used = 200` (one shared initial generation per pair). Analysis `missingness_summary` reports `complete_pairs = 200` with zero failed or unscorable episodes. Deterministic reconciliation records `hosted_model_call_event_count = 200`, `completed_hosted_model_call_event_count = 200`, `cache_miss_count = 200`, and `stage_counts = {"shared_initial_generation": 200}`; `cross_condition_cache_key_reuse_observed = false` and all deterministic consistency checks passed.

Primary binary outcomes and contingency. The validator mapped each episode to a binary executable outcome (`validator.valid`). Condition summaries show `baseline_successes = 200/200` and `guarded_successes = 200/200` (`success_rate = 1.0` for both arms). The paired 2×2 contingency table is $n_{11} = 200$, $n_{10} = 0$, $n_{01} = 0$, $n_{00} = 0$; exact McNemar two-sided $p = 1.0$. The paired bootstrap (`seed = 1729`; 10,000 resamples) produced a percentile 95% CI for the paired difference of $[0.0, 0.0]$. These files are archived (`analysis/condition_summary.csv` and `analysis/paired_contingency_table.csv`) and exactly reproduce the contingency and CI numbers reported here.

Repair activity, validator diagnostics, and representative examples. Across the confirmatory sample the validator flagged no violations on any shared initial candidate; as a result `repair_attempt_count = 0` and `repair_applied_count = 0`. Representative per-episode scoring JSON artifacts illustrate the pattern. For example, task-000004 (`pair_id` `pair-000004`) `shared_initial_candidate` and both condition responses were identical:

```
interface edge2 admin down interface edge2 vlan 40 interface edge2 admin up interface edge1 admin down
```

For task-000004 the archived validator trace shows `parsed_command_count = 4`, `required_command_count = 4`, `observed_touched_field_count = 3`, and `validation_after.valid = true`. Similarly, task-000005 shows `parsed_command_count = 3` and `required_command_count = 3` with `validation_after.valid = true`; task-000006 shows `parsed_command_count = 6` and `required_command_count = 6` with `validation_after.valid = true`. These per-episode diagnostics (`execution/scoring/task-00000X-*.json`) demonstrate that the validator parsed the expected command set, matched required sequence checks, and reported no violation codes for the sampled candidates. Because the validator never produced a violation, the guarded repair path was never invoked and could not alter any outcome.

Contamination and provider audit summary. Preexecution contamination checks found no exact-match contamination exclusions; `analysis/contamination_summary.csv` is empty. Provider audit and execution manifests corroborate the sampling semantics: `hosted_model_call_event_count = 200`, `cache_miss_count = 200`, and `stage_counts` indicating exactly one `shared_initial_generation` per pair. No infrastructure retries or failures occurred; `analysis/analysis_log.jsonl` documents the paired analysis completion and bootstrap parameters (`bootstrap_resamples = 10000`; `bootstrap_seed = 1729`).

Compact repair summary (explicit). In concise preregistered terms: `total_pairs = 200`; `repair_attempt_count = 0`; `repair_applied_count = 0`. The empty repair counts fully explain the degenerate paired contingency (all pairs valid before any repair opportunity). All artifacts supporting these counts are archived in the evidence bundle (notably the `execution/scoring` JSONs, `execution/task_manifest.jsonl`, `analysis/paired_contingency_table.csv`, and `analysis/condition_summary.csv`) and permit independent verification that no repairs were produced or applied in guarded episodes.

V. DISCUSSION

Interpreting the executed estimand. The `shared_initial_candidate` semantics were intentionally chosen to isolate the causal effect of deterministic validation and any permitted repair acting on the exact same generator output. The executed estimand therefore answers a narrowly defined causal question: when baseline and guarded episodes begin from the identical sampled candidate, can deterministic validation and at most one automated repair change the validator-valid/executable outcome? The observed null effect (paired difference = 0.0) shows that, for this task bank and this single sampled candidate per pair, the validator found no violations and the permitted repair path was never exercised. Importantly, this does not evaluate whether guarded pipelines reduce first-pass generator error rates under independent sampling or different model temperatures; such claims would require a different preregistered estimand and execution.

Artifact-level diagnostics explaining the degenerate outcome. Multiple archived artifacts explain why the guarded path was not exercised. Representative per-episode scorer JSONs under `execution/scoring/` (for example task-000004, task-000005, task-000006) show that the `shared_initial_candidate` content exactly satisfied the recorded task `required_sequence` and `workflow_policies`: `parsed_command_count` equaled `required_command_count` in these examples and `validation_after.normalized_configuration` matched the reference `expected_final_state`. The validator traces consistently set `validator.valid = true` and `repair_applied = false` for the sampled examples, and `analysis/condition_summary.csv` reports 200 successes in each condition. Provider audit artifacts show `hosted_model_call_event_count = 200` and `cache_miss_count = 200`, confirming that each pair used a single freshly returned initial generation (shared) rather than multiple independent samples. These concrete, archived observations together account for the absence of discordant pairs.

Statistical and inferential implications. From a statistical standpoint the complete absence of discordant pairs ($n_0 = n_1 = 0$) makes the confirmatory McNemar test degenerate: there is no observed evidence that the validator or repair step altered any outcome under the executed estimand. This outcome also limits the inferential information the design can provide about repair efficacy; power calculations conducted pre-execution assumed a nonzero baseline failure rate and thus cannot be used post-hoc to infer a repair effect here. The artifact record supports this limitation explicitly (`analysis/paired_contingency_table.csv` and `analysis/analysis_log.jsonl`). Consequently, meaningful estimation of repair benefit requires a design that induces or selects for nontrivial validator failure rates (see preregistered follow-on designs below).

Design-mechanism interpretation without overreach. It is important to avoid two incorrect readings of the result. First, the null paired difference does not imply that validator-guided repair is unnecessary generally; it only shows that for the

executed estimand—one shared sampled candidate per intent drawn from the provided deterministic task bank—the validator detected no faults and therefore repair could not act. Second, the result does not imply that sampling nondeterminism was absent: `model_configuration.json` records `temperature = null/unset` and, as we explicitly note, `null/unset` is not evidence of deterministic sampling and does not imply `temperature = 0`. The single shared initial generation per pair was the artifact used in both arms; nondeterministic sampling could still affect a different estimand that samples independently per condition.

Practical lessons for future preregistered experiments. The archived execution suggests concrete, preregistered modifications to exercise the guarded repair path: (1) remove `shared_initial_candidate` semantics so baseline and guarded arms receive independent first-pass samples (this allows repairs to act on generator faults but requires a revised analysis plan to separate generator variability from repair effects); (2) select or synthesize tasks annotated as "extreme" difficulty or inject adversarial perturbations to raise expected validator failure rates and thereby exercise repair logic (the task manifest includes difficulty annotations that can be used deterministically); (3) preregister non-null temperature sweeps to increase sample diversity and raise the probability of first-pass faults; (4) increase validator fidelity by integrating vendor CLI semantics or packet-level emulation so repairs are evaluated against dynamic runtime behavior rather than only logical workflow constraints; and (5) broaden the model scope beyond a single declared generator to assess whether guard-repair interactions generalize across generator families. All of these modifications are consistent with the preregistered follow-on designs archived alongside the execution artifacts and do not require inventing new infrastructure beyond what is already documented.

Validator and task-bank interactions as a methodological point. The present artifacts show that a deterministic task generator combined with an LLM generator configured under the executed semantics can produce many valid candidates for constrained configuration intents—particularly when the task specification includes a concise `required_sequence` and the validator enforces deterministic workflow checks. This observation highlights a methodological interaction: the joint distribution induced by (task generator $\times$ sampling semantics $\times$ model configuration $\times$ validator fidelity) determines whether a guarded repair path will be invoked. Careful preregistered control of each of those factors is therefore necessary to produce a confirmatory test of repair efficacy rather than, as here, a test of downstream validator behavior on already-valid candidates.

Reproducibility, auditability, and limits of evidence. The execution and analysis artifacts (`execution/raw_results.jsonl`, `execution/task_manifest.jsonl`, `execution/scoring/*.json`, `analysis/*.csv`, `model_configuration.json`) provide machine-verifiable traces that reproduce the executed estimand and demonstrate repair non-occurrence. Preexecution contamination checks produced no exact-match exclusions. Deterministic reconciliation and provider audit artifacts

show consistent episode accounting and one shared initial generation per pair. Nevertheless, external validity is limited: the task bank is synthetic, the validator is an abstraction that does not execute vendor CLI or perform packet-level emulation, and the run used a single declared generator and hosted adapter. These constraints are documented in the limitations section and should guide interpretation.

Summary takeaway. The artifact-grounded evidence is internally consistent and supports a precise, bounded conclusion: under the preregistered `shared_initial_candidate` semantics and the executed synthetic task bank, deterministic validation found no violations and the permitted automated repair path was never exercised, yielding identical valid outcome proportions in baseline and guarded arms. This narrow, reproducible finding clarifies an estimand boundary and motivates preregistered extensions that intentionally increase generator-level faults or validator fidelity to measure repair utility in operationally challenging regimes.

VI. LIMITATIONS

- `Shared_initial_candidate` semantics: the design produced exactly one sampled initial generation per intent and reused it across baseline and guarded conditions; this measures validator/repair effects only and cannot detect differences caused by independent per-condition sampling or prompt engineering.
- Temperature unset (temperature = null) in the archived model configuration: null/unset is not evidence of deterministic sampling and does not imply temperature = 0; sampling nondeterminism may still have occurred during the single sampled generation.
- Single model and single hosted provider: results reflect gpt-5-mini under the archived adapter; generalization to other models or model families is unsupported by these artifacts.
- Synthetic task bank and validator abstraction: tasks were synthetic 'intent_configuration_repair_v1' items and the deterministic validator is an abstraction; realistic vendor CLI idiosyncrasies, emergent runtime interactions, and hardware effects were not executed on live devices.
- No observed validator failures: all initial candidates were validator-valid, so the experiment could not assess the repair step's effectiveness; the null effect therefore reflects the task bank and sampling semantics rather than evidence that GVR is unnecessary in general.

VII. CONCLUSION

We executed a fully preregistered paired confirmatory experiment that measures the downstream effect of a deterministic validator plus an allowed single automated repair on the exact same sampled LLM candidate per intent (`shared_initial_candidate` semantics). Using the declared generator gpt-5-mini and deterministic_netops_validator_v5 on 200 paired intents, every shared initial candidate passed validation and no repairs were attempted or applied; guarded and baseline outcomes were identical for the executed estimand

(paired difference = 0.0; McNemar $p = 1.0$; paired bootstrap CI = [0.0, 0.0]). This artifact-grounded null result demonstrates an estimand boundary: under `shared_initial_candidate` semantics and the executed task bank the guarded pipeline could not be exercised and therefore could not change outcomes. It does not imply that GVR pipelines are unnecessary generally. Preregistered follow-on experiments that (1) remove `shared_initial_candidate` semantics, (2) increase task difficulty or inject adversarial inputs, (3) set explicit sampling parameters, and (4) raise validator fidelity are required to empirically evaluate repair efficacy under realistic sampling variability and adversarial conditions. The archived artifacts and reconciliation files enable exact reproduction of the executed estimand and provide a secure base for precise preregistered extensions.

References [1] A. Alansari and H. Luqman, "Large Language Models Hallucination: A Comprehensive Survey," arXiv preprint arXiv:2510.06265, 2025. [2] B. Zhang, H. Rao, and D. Zhao, "Evidence-Grounded RAG for Cloud-Native DevOps: Hallucination-Resistant AIOps Question Answering over Private Operations Documents," J. Autom. Cloud-Native Syst., 2024. [3] P. Jana et al., "TerraFormer: Automated Infrastructure-as-Code with LLMs Fine-Tuned via Policy-Guided Verifier Feedback," Proc. ACM Symp. (2026). [4] Y. Miao et al., "An Empirical Study of NetOps Capability of Pre-Trained Large Language Models," arXiv:2309.05557, 2023. [5] L. Zhang et al., "A Survey of AIOps in the Era of Large Language Models," ACM Comput. Surv., 2025.

One-line reiteration of the primary estimand limit: the preregistered primary estimand intentionally used `shared_initial_candidate` semantics; evaluating per-condition independent sampling requires a different preregistration and analysis plan (explicitly recommended above).

DISCLOSURE STATEMENT

Disclosure (concise): The human Research Director selected the broad topic family (Generative AI / LLMs for NetOps) before the autonomy lock. After the autonomy lock the autonomous system performed literature review, experiment selection, preregistration (PREREG-CAND-001-VER-GVR-2026-09-01), code generation, execution, deterministic validation, automated analysis, and manuscript drafting without manual human editing of technical content. Declared model used in execution: gpt-5-mini via hosted adapter 'openai_responses' (execution used `model_calls_used` = 200; archived `model_configuration.json` records temperature = null/unset; null/unset is not evidence of deterministic sampling and does not imply temperature = 0). Generation semantics executed: `shared_initial_candidate` — exactly one sampled initial generation per intent was produced and deterministically reused for both baseline and guarded conditions. Validator: `deterministic_netops_validator_v5` scored candidates using 'full_intent_constraint_validation' and a single automated repair iteration was permitted in guarded episodes; in this run the validator flagged no violations and no repairs were applied (`repair_attempt_count`

= 0; repair_applied_count = 0). Execution accounting: completed_episode_count = 400 (200 paired intents) completed without preregistration deviations. Master prompt immutable reference: provenance/master_prompt.txt — SHA-256: 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15ba. Pre-lock infrastructure development and rehearsal occurred but pre-lock artifacts were not used as empirical evidence in this final run. After the autonomy lock no human manually rewrote technical results, manuscript text, figures, or references.

REFERENCES

- [1] Aisha Alansari, Hamzah Luqman, "Large Language Models Hallucination: A Comprehensive Survey", 2025, 10.48550/arxiv.2510.06265
- [2] Boning Zhang, Hengning Rao, Derek Zhao , "Evidence-Grounded RAG for Cloud-Native DevOps: Hallucination-Resistant AIOps Question Answering over Private Operations Documents", 2024, 10.69987/jacs.2024.40308
- [3] Prithwish Jana, Sam Davidson, Bhavana Bhasker, Andrey Kan, Anoop Deoras, Laurent Callot, "TerraFormer: Automated Infrastructure-as-Code with LLMs Fine-Tuned via Policy-Guided Verifier Feedback", 2026, 10.1145/3786583.3786898
- [4] Yukai Miao, Yu Bai, Li Chen, Dan Li, Haifeng Sun, Xizheng Wang, Ziqiu Luo, Ren, Yanyu, Dapeng Sun, Xiuting Xu, Qi Zhang, Chao Xiang, Xinchu Li, "An Empirical Study of NetOps Capability of Pre-Trained Large Language Models", 2023, 10.48550/arxiv.2309.05557
- [5] Lingzhe Zhang, Tong Jia, Mengxi Jia, Yifan Wu, Aiwei Liu, Yong Yang, Zhonghai Wu, Xuming Hu, Philip S. Yu, Ying Li, "A Survey of AIOps in the Era of Large Language Models", 2025, 10.1145/3746635